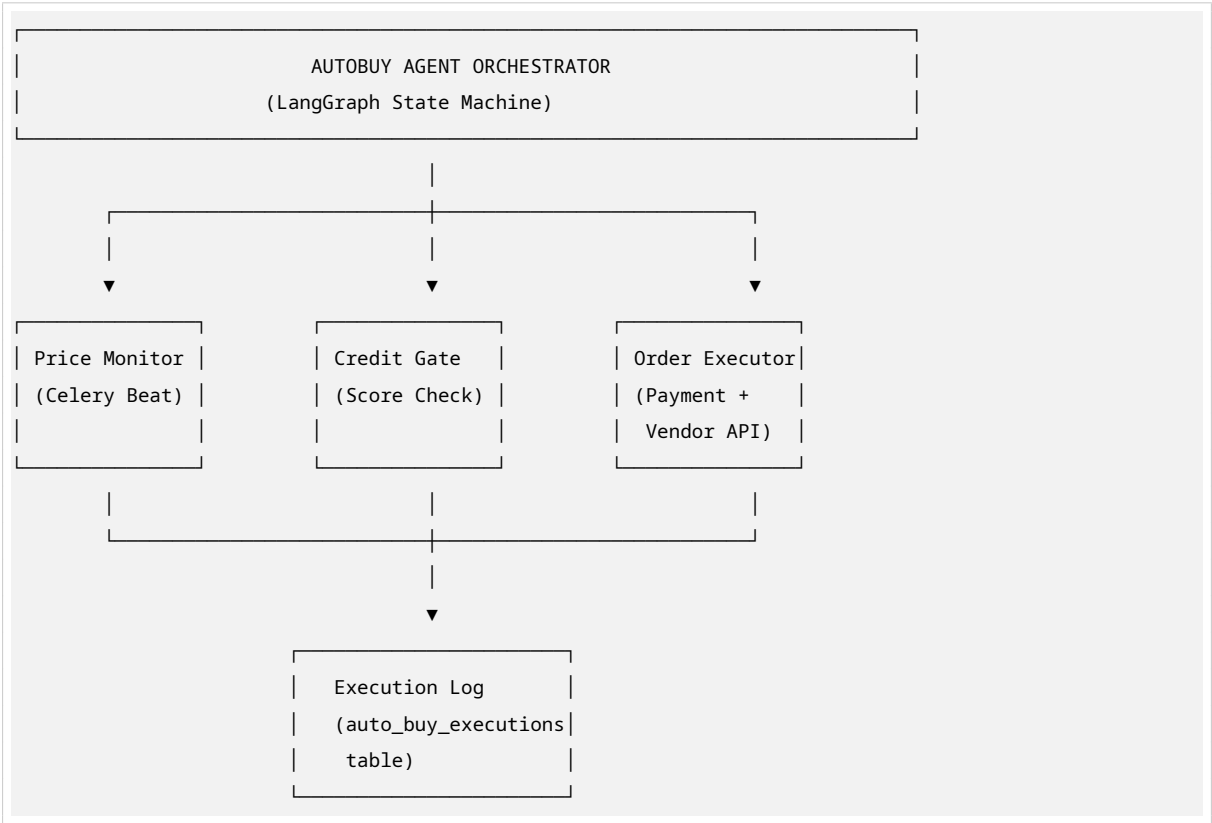


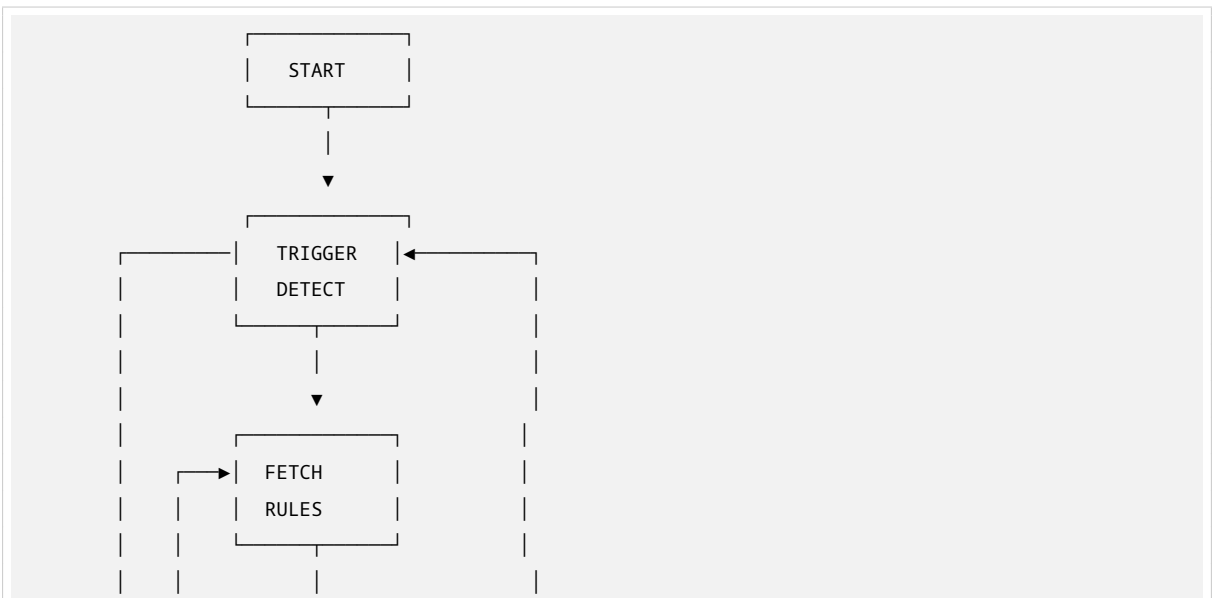
PCE AutoBuy Agent — Architecture & Implementation

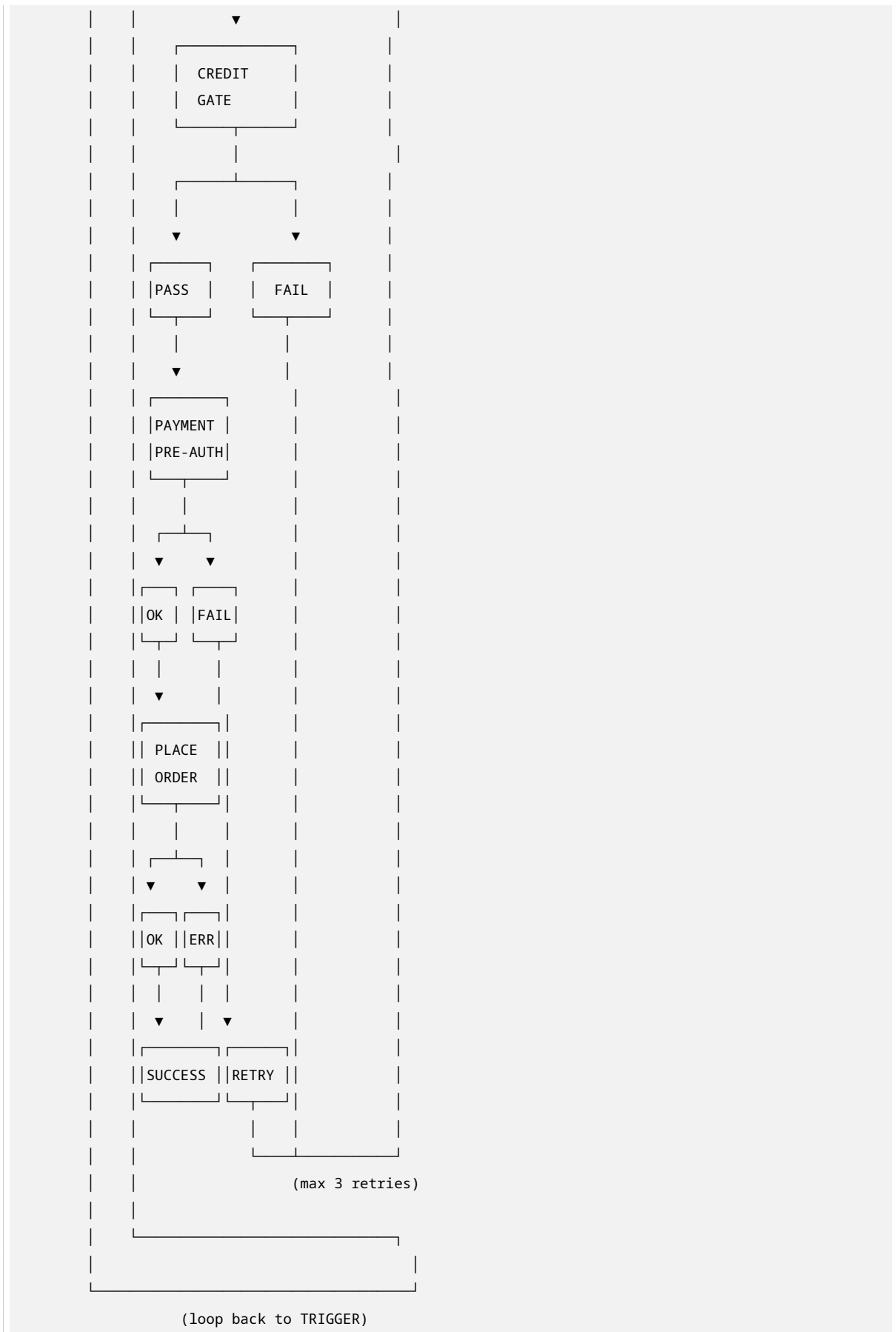
Purpose: Autonomous purchasing agent that monitors prices, evaluates user creditworthiness, and places orders on behalf of users when trigger conditions are met.

1. Architecture Overview



2. LangGraph State Machine





3. State Definitions

```
from typing import TypedDict, Optional, List
from datetime import datetime

class AutoBuyState(TypedDict):
    """LangGraph state for AutoBuy execution."""

    # Input
    rule_id: str
    user_id: str
    product_id: str
    trigger_type: str # 'price_drop', 'restock', 'scheduled'
    trigger_price: float

    # Fetched data
    rule: Optional[dict]
    user_profile: Optional[dict]
    credit_profile: Optional[dict]
    offer: Optional[dict]
    payment_method: Optional[dict]
    address: Optional[dict]

    # Gate results
    credit_check_passed: Optional[bool]
    credit_check_reason: Optional[str]

    # Payment
    payment_authorized: Optional[bool]
    payment_txn_id: Optional[str]

    # Order
    order_placed: Optional[bool]
    vendor_order_id: Optional[str]
    order_id: Optional[str]

    # Error handling
    error_code: Optional[str]
    error_message: Optional[str]
    retry_count: int

    # Final status
    status: str # 'initiated', 'credit_check_failed', 'payment_failed',
               # 'order_placed', 'vendor_error', 'success', 'cancelled'
    completed_at: Optional[datetime]
```

4. Execution Flow Detail

Step 1: Trigger Detection (Celery Beat)

```
# Every 5 minutes, Celery Beat triggers:
@celery_app.task
def scan_auto_buy_triggers():
    """
    Scans for AutoBuy rule triggers:
    1. Price drops below trigger_price
    2. Products back in stock
    3. Scheduled triggers (e.g., "buy on Diwali sale")
    """
    db = SessionLocal()
    try:
        # Find active rules where current price <= trigger_price
        triggered_rules = db.execute(text("""
            SELECT abr.*, vo.current_price, vo.in_stock
            FROM auto_buy_rules abr
            JOIN vendor_offers vo ON abr.product_id = vo.product_id
            WHERE abr.is_active = TRUE
            AND (abr.expiry_date IS NULL OR abr.expiry_date >= CURRENT_DATE)
            AND (
                (abr.trigger_price IS NOT NULL AND vo.current_price <= abr.trigger_price)
                OR (abr.trigger_drop_pct IS NOT NULL AND
                    vo.current_price <= (SELECT price FROM price_history
                                        WHERE offer_id = vo.id
                                        ORDER BY recorded_at DESC LIMIT 1)
                    * (1 - abr.trigger_drop_pct/100))
            )
            AND vo.in_stock = TRUE
        """)).fetchall()

        for rule in triggered_rules:
            # Queue AutoBuy execution
            execute_auto_buy.delay(rule.id)

            # Update trigger count
            db.execute(text("""
                UPDATE auto_buy_rules
                SET times_triggered = times_triggered + 1,
                    last_triggered_at = NOW()
                WHERE id = :id
            """), {"id": rule.id})

        db.commit()

    finally:
        db.close()
```

Step 2: Credit Gate

```
def credit_gate(state: AutoBuyState) -> AutoBuyState:
    """
    Validates user creditworthiness before proceeding.
    Hard gates — any failure aborts the workflow.
    """

    from app.tasks_credit_scoring import update_user_credit_score

    # Ensure score is fresh (real-time update)
    update_user_credit_score.delay(state["user_id"])

    db = SessionLocal()
    try:
        # Fetch latest credit profile
        credit = db.execute(text("""
            SELECT trust_score, auto_buy_eligible, is_flagged, kyc_boost_score
            FROM user_credit_profiles
            WHERE user_id = :user_id
        """), {"user_id": state["user_id"]}).fetchone()

        if not credit:
            return _fail(state, "CREDIT_NO_PROFILE", "No credit profile found")

        trust_score, eligible, flagged, kyc = credit

        # Gate 1: Eligibility
        if not eligible:
            return _fail(state, "CREDIT_NOT_ELIGIBLE",
                f"Trust score {trust_score}, eligible={eligible}")

        # Gate 2: Flagged
        if flagged:
            return _fail(state, "CREDIT_FLAGGED", "Account is flagged")

        # Gate 3: Limits
        user_prof = db.execute(text("""
            SELECT max_auto_buy_order_value, daily_auto_buy_limit, monthly_auto_buy_limit
            FROM user_profiles WHERE user_id = :user_id
        """), {"user_id": state["user_id"]}).fetchone()

        today_spent = db.execute(text("""
            SELECT COALESCE(SUM(order_value), 0) FROM purchase_orders
            WHERE user_id = :user_id AND triggered_by = 'auto_buy_agent'
            AND placed_at >= CURRENT_DATE
        """), {"user_id": state["user_id"]}).scalar()

        month_spent = db.execute(text("""
            SELECT COALESCE(SUM(order_value), 0) FROM purchase_orders
            WHERE user_id = :user_id AND triggered_by = 'auto_buy_agent'
            AND placed_at >= DATE_TRUNC('month', CURRENT_DATE)
        """), {"user_id": state["user_id"]}).scalar()
```

```

"""), {"user_id": state["user_id"]}).scalar()

offer_price = state["trigger_price"]

if offer_price > user_prof.max_auto_buy_order_value:
    return _fail(state, "LIMIT_ORDER_VALUE",
                  f"₹{offer_price} > max ₹{user_prof.max_auto_buy_order_value}")

if today_spent + offer_price > user_prof.daily_auto_buy_limit:
    return _fail(state, "LIMIT_DAILY",
                  f"Daily limit ₹{user_prof.daily_auto_buy_limit} exceeded")

if month_spent + offer_price > user_prof.monthly_auto_buy_limit:
    return _fail(state, "LIMIT_MONTHLY",
                  f"Monthly limit ₹{user_prof.monthly_auto_buy_limit} exceeded")

# Gate 4: Velocity
recent_orders = db.execute(text("""
    SELECT COUNT(*) FROM purchase_orders
    WHERE user_id = :user_id AND triggered_by = 'auto_buy_agent'
    AND placed_at >= NOW() - INTERVAL '24 hours'
"""), {"user_id": state["user_id"]}).scalar()

if recent_orders >= 5:
    return _fail(state, "VELOCITY_LIMIT", "5+ AutoBuy orders in 24h")

# Gate 5: Consent
consent = db.execute(text("""
    SELECT consent_given FROM user_consents
    WHERE user_id = :user_id AND consent_type = 'auto_buy'
    ORDER BY created_at DESC LIMIT 1
"""), {"user_id": state["user_id"]}).fetchone()

if not consent or not consent[0]:
    return _fail(state, "NO_CONSENT", "User has not consented to AutoBuy")

# All gates passed
state["credit_check_passed"] = True
state["credit_check_reason"] = "All gates passed"
state["credit_profile"] = {
    "trust_score": trust_score,
    "kyc_boost": kyc,
}

return state

finally:
    db.close()

def _fail(state: AutoBuyState, code: str, message: str) -> AutoBuyState:

```

```

state["credit_check_passed"] = False
state["credit_check_reason"] = message
state["error_code"] = code
state["error_message"] = message
state["status"] = "credit_check_failed"
return state

```

Step 3: Payment Pre-Authorization

```

def payment_preauth(state: AutoBuyState) -> AutoBuyState:
    """
    Pre-authorizes payment before placing the vendor order.
    This holds funds but doesn't capture until order is confirmed.
    """
    db = SessionLocal()
    try:
        # Get preferred payment method
        pm = db.execute(text("""
            SELECT * FROM user_payment_methods
            WHERE id = :pm_id OR (user_id = :user_id AND is_default = TRUE AND is_active = TRUE)
            LIMIT 1
        """), {
            "pm_id": state["rule"].get("preferred_payment_method_id"),
            "user_id": state["user_id"]
        }).fetchone()

        if not pm:
            return _payment_fail(state, "NO_PAYMENT_METHOD", "No active payment method")

        state["payment_method"] = dict(pm)

        # Call payment gateway (Razorpay example)
        # In production, use the gateway SDK
        gateway_response = _call_payment_gateway(
            gateway=pm.gateway,
            token=pm.gateway_token,
            amount=state["trigger_price"],
            currency="INR",
            capture=False, # Authorize only, don't capture yet
        )

        if gateway_response["status"] == "authorized":
            state["payment_authorized"] = True
            state["payment_txn_id"] = gateway_response["id"]

        # Record transaction
        db.execute(text("""
            INSERT INTO payment_transactions
            (order_id, user_id, payment_method_id, amount, currency, gateway,
             gateway_txn_id, gateway_status, status, created_at)
            VALUES (:order_id, :user_id, :pm_id, :amount, 'INR', :gateway,

```

```

        :txn_id, 'authorized', 'pending', NOW())
    """), {
      "order_id": state.get("order_id"),
      "user_id": state["user_id"],
      "pm_id": pm.id,
      "amount": state["trigger_price"],
      "gateway": pm.gateway,
      "txn_id": gateway_response["id"],
    })
    db.commit()

  else:
    return _payment_fail(state, "AUTH_FAILED", gateway_response.get("error", "Unknown"))

  return state

except Exception as e:
  db.rollback()
  return _payment_fail(state, "PAYMENT_ERROR", str(e))
finally:
  db.close()

def _payment_fail(state: AutoBuyState, code: str, message: str) -> AutoBuyState:
  state["payment_authorized"] = False
  state["error_code"] = code
  state["error_message"] = message
  state["status"] = "payment_failed"
  return state

```

Step 4: Order Placement

```

def place_order(state: AutoBuyState) -> AutoBuyState:
    """
    Places the actual order with the vendor.
    This is vendor-specific — each vendor may have a different API.
    """
    db = SessionLocal()
    try:
        # Get offer details
        offer = db.execute(text("""
            SELECT vo.*, v.name as vendor_name, v.domain
            FROM vendor_offers vo
            JOIN vendors v ON vo.vendor_id = v.id
            WHERE vo.id = :offer_id
        """), {"offer_id": state["offer"]["id"]}).fetchone()

        # Get shipping address
        address = db.execute(text("""
            SELECT * FROM user_addresses
            WHERE id = :addr_id OR (user_id = :user_id AND is_default = TRUE)
        """), {"addr_id": state["address_id"], "user_id": state["user_id"]}).fetchone()
    except Exception as e:
        return _payment_fail(state, "ORDER_PLACEMENT_ERROR", str(e))
    finally:
        db.close()

    return state

```



```

        LIMIT 1
    """), {
        "addr_id": state["rule"].get("preferred_address_id"),
        "user_id": state["user_id"]
    }).fetchone()

if not address:
    return _order_fail(state, "NO_ADDRESS", "No shipping address found")

state["address"] = dict(address)

# Vendor-specific order placement
# This is a placeholder — each vendor integration is different
vendor_order = _call_vendor_api(
    vendor_domain=offer.domain,
    product_url=offer.product_url,
    quantity=state["rule"].get("max_quantity", 1),
    shipping_address={
        "name": address.full_name,
        "phone": address.phone,
        "line1": address.address_line1,
        "line2": address.address_line2,
        "city": address.city,
        "state": address.state,
        "postal": address.postal_code,
    },
    payment_token=state["payment_method"]["gateway_token"],
)

if vendor_order["status"] == "success":
    state["order_placed"] = True
    state["vendor_order_id"] = vendor_order["order_id"]
    state["status"] = "order_placed"

# Create purchase order record
order_result = db.execute(text("""
    INSERT INTO purchase_orders
    (user_id, offer_id, product_id, vendor_id, order_value, currency,
    quantity, status, payment_method_id, payment_gateway,
    payment_gateway_txn_id, vendor_order_id, triggered_by, auto_buy_rule_id,
    placed_at, created_at)
    VALUES (:user_id, :offer_id, :product_id, :vendor_id, :value, 'INR',
            :qty, 'order_placed', :pm_id, :gateway,
            :txn_id, :vendor_order_id, 'auto_buy_agent', :rule_id,
            NOW(), NOW())
    RETURNING id
"""), {
    "user_id": state["user_id"],
    "offer_id": offer.id,
    "product_id": state["product_id"],
    "vendor_id": offer.vendor_id,

```

```

        "value": state["trigger_price"],
        "qty": state["rule"].get("max_quantity", 1),
        "pm_id": state["payment_method"]["id"],
        "gateway": state["payment_method"]["gateway"],
        "txn_id": state["payment_txn_id"],
        "vendor_order_id": vendor_order["order_id"],
        "rule_id": state["rule_id"],
    }).fetchone()

    state["order_id"] = str(order_result[0])

    # Update payment transaction with order_id
    db.execute(text("""
        UPDATE payment_transactions
        SET order_id = :order_id, status = 'success'
        WHERE gateway_txn_id = :txn_id
    """), {"order_id": state["order_id"], "txn_id": state["payment_txn_id"]})

    # Capture payment (move from authorized to captured)
    _capture_payment(state["payment_txn_id"])

    db.commit()

    # Send notification to user
    _notify_user_auto_buy_success(state)

else:
    return _order_fail(state, "VENDOR_ERROR", vendor_order.get("error", "Unknown"))

return state

except Exception as e:
    db.rollback()
    return _order_fail(state, "ORDER_EXCEPTION", str(e))
finally:
    db.close()

def _order_fail(state: AutoBuyState, code: str, message: str) -> AutoBuyState:
    state["order_placed"] = False
    state["error_code"] = code
    state["error_message"] = message
    state["status"] = "vendor_error"

    # Release payment authorization
    if state.get("payment_authorized") and state.get("payment_txn_id"):
        _void_payment(state["payment_txn_id"])

    return state

```

5. Retry Logic

```
def should_retry(state: AutoBuyState) -> bool:
    """Determine if the workflow should retry."""
    retryable_errors = ["VENDOR_ERROR", "PAYMENT_ERROR", "AUTH_FAILED"]

    if state["error_code"] not in retryable_errors:
        return False

    if state["retry_count"] >= 3:
        return False

    # Don't retry if price has changed significantly
    current_price = _get_current_price(state["offer"]["id"])
    if current_price > state["trigger_price"] * 1.05: # >5% price increase
        return False

    return True

def retry_with_backoff(state: AutoBuyState) -> AutoBuyState:
    """Increment retry and schedule next attempt."""
    state["retry_count"] += 1
    delay = 60 * (2 ** state["retry_count"]) # 2min, 4min, 8min

    execute_auto_buy.apply_async(
        args=[state["rule_id"]],
        countdown=delay,
    )

    state["status"] = "retry_scheduled"
    return state
```

6. Monitoring & Observability

Grafana Metrics to Track

Metric	Query	Alert Threshold
AutoBuy success rate	<code>rate(auto_buy_executions[status='order_placed'] [1h]) / rate(auto_buy_executions[1h])</code>	<code>< 95%</code>
Credit gate rejections	<code>rate(auto_buy_executions[status='credit_check_failed'] [1h])</code>	<code>> 0%</code>
Payment failures	<code>rate(auto_buy_executions[status='payment_failed'] [1h])</code>	<code>> 5%</code>
Avg execution time	<code>histogram_quantile(0.95, auto_buy_execution_duration_seconds)</code>	<code>> 30s</code>
Queue depth	<code>celery_queue_length{queue='autobuy'}</code>	<code>> 100</code>

Logging Strategy

Every AutoBuy execution writes to `auto_buy_executions` table:

- `initiated` → `credit_check_passed` → `payment_authorized` → `order_placed` → `success`
 - Or any failure state with `error_code` and `error_message`
-

7. Security Considerations

Risk	Mitigation
Payment token theft	Tokens stored only in DB, never logged. Gateway tokens are revocable.
Order manipulation	All prices verified against DB at execution time, not from trigger params.
Duplicate orders	UNIQUE constraint on (<code>user_id</code> , <code>product_id</code> , <code>DATE(placed_at)</code>) for AutoBuy.
Vendor API abuse	Rate limiting per vendor domain. Circuit breaker on vendor errors.
User impersonation	JWT + device hash validation on every AutoBuy rule creation.
Price spike exploitation	Current price re-verified at execution time. Abort if >5% above trigger.

8. Implementation Checklist

- Run `migration_v2_to_v3_autobuy.sql` to create all tables
- Implement `tasks_credit_scoring.py` nightly batch
- Add Celery Beat schedule for credit scoring
- Implement `tasks_autobuy.py` with LangGraph workflow
- Integrate payment gateway (Razorpay recommended for India)
- Implement vendor order APIs (Amazon, Flipkart affiliate APIs)
- Build user consent flow in Flutter app
- Add AutoBuy rule creation UI in Flutter app
- Configure Grafana dashboards for AutoBuy metrics
- Write integration tests for full workflow
- Load test: simulate 1000 concurrent AutoBuy triggers